Word & Character Counter

Project Report • Python + Flask • Render Deployment

1. Project Overview

This project is a small Flask web application that accepts a paragraph and calculates the number of characters, words, sentences, and spaces. It was designed to practice Python string manipulation and basic text processing.

2. Objectives

• Practice Python string operations.
• Build a simple interactive text-analysis application.
• Handle empty input safely.
• Test the application with automated unit tests.
• Prepare the application for GitHub and Render deployment.

3. Key Features

Feature	Implementation
Character count	len(text)
Word count	len(text.split())
Sentence count	Regular expression for ., ! and ?
Space count	text.count(" ")
Empty input	Returns zero statistics
Web UI	Responsive HTML/CSS interface
Testing	Python unittest
Deployment	Render Web Service + Gunicorn

4. Technology Stack

Python 3.13.5, Flask 3.1.2, Gunicorn 23.0.0, HTML5, CSS3, Git/GitHub, and Render.

5. Application Workflow

1) User enters a paragraph. 2) Flask receives the submitted text. 3) The analyze_text() function performs the calculations. 4) Results are rendered back into the page. 5) The user can clear the form and analyze another paragraph.

6. Core Logic

```
len(text)                        # characters
len(text.split())               # words
len(re.findall(r"[.!?]+", text)) # sentences
text.count(" ")                 # spaces
```

The sentence counter is intentionally a basic educational implementation. It counts runs of sentence-ending punctuation rather than attempting advanced natural-language sentence detection.

7. Testing

The included unittest suite covers empty input, basic and multiple-sentence analysis, the GET page, and POST processing. Run: python -m unittest discover -s tests -v

8. Deployment

Render is configured as a Python Web Service. Build command: pip install -r requirements.txt. Start command: gunicorn app:app. The repository includes render.yaml so the deployment configuration is version-controlled.

9. GitHub & Rollback

The repository contains a README with PowerShell setup instructions and docs/rollback-evidence.md describing how to create a known-good v1.0.0 tag, identify a bad commit, and use git revert or a known-good commit for recovery. Actual rollback evidence should only be claimed after it has been performed and screenshotted.

10. Submission Checklist

Public GitHub repository • README.md • requirements.txt • render.yaml • tests • rollback documentation • successful local tests • successful Render deployment • live URL screenshot • GitHub screenshot • rollback evidence screenshot when performed • repository URL submitted through the internship portal • LinkedIn post published.

11. References

Render Flask deployment guide: https://render.com/docs/deploy-flask
Render first deployment: https://render.com/docs/your-first-deploy
GitHub command-line guide: https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github